



TheAgenticAIEngineer ✓



LLM & FINE-TUNING

— with —

PEFT: LORA & QLORA

(COMPLETE NOTES)



Master Efficient Model Adaptation:
Less VRAM, Less Training



Base Model

Start with a large,
pre-trained LLM
(e.g., Llama, Mistral,
GPT, etc.)



Adaptation

Add sparse,
low-rank adapter
layers (LoRA / QLoRA)



Fine-Tuned LLM

Deploy an efficient,
task-specific model

"Accessible LLM Customization - For All!"



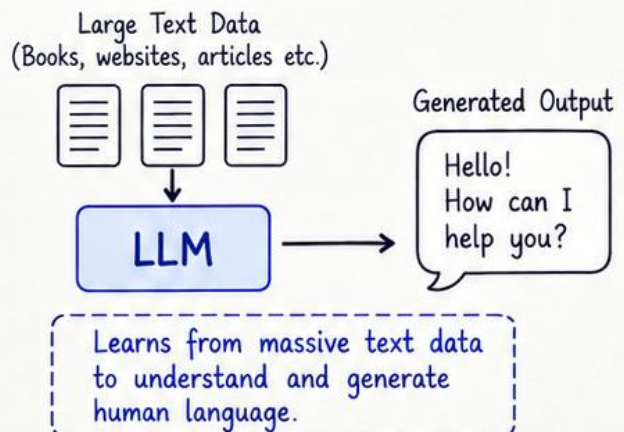


LLM Fundamentals (Part 1)



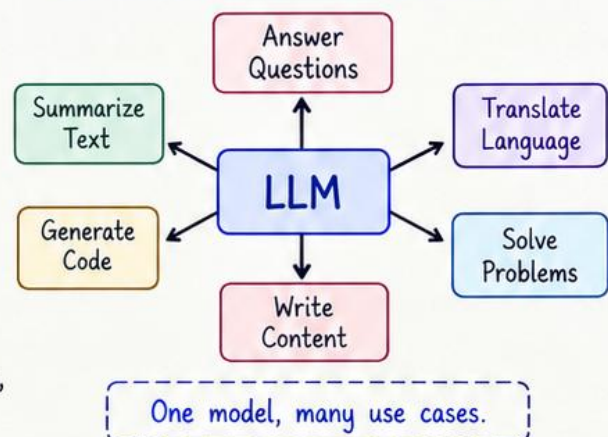
1 What is an LLM?

- An LLM (Large Language Model) is a type of AI model trained on **massive text data** to understand, generate and work with **human language**.
- It learns **patterns, context and meaning** from text and can perform many tasks like answering questions, summarizing, translating, writing, etc.
- **Examples:**
GPT, LLaMA, Claude, Gemini, Mistral.



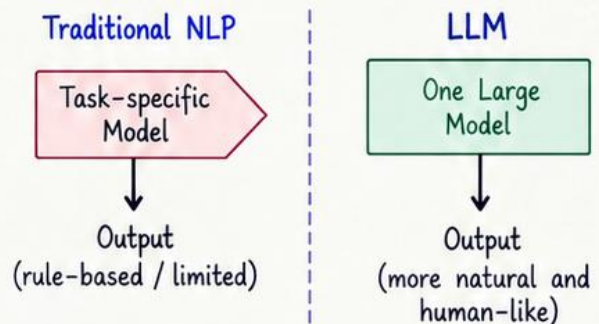
2 Why we need LLM?

- Traditional NLP models were good at specific tasks but had **limited understanding of context and meaning**.
- LLMs can understand natural language, reason, generate human-like responses and work across many different tasks with **one model**.
- They save time and effort and can be used in real-world applications.
- **Examples:** Chatbots, content generation, translation, code generation, research, education etc.



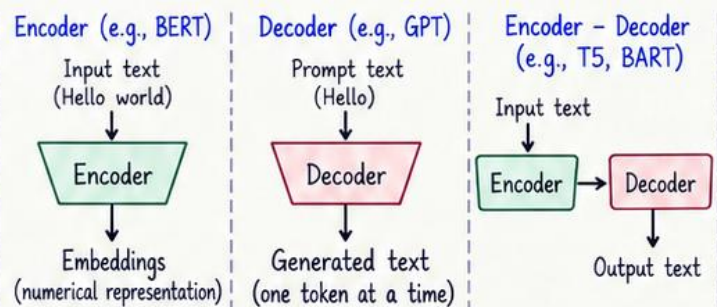
3 LLM vs Traditional NLP

- Traditional NLP uses smaller, task-specific models with **limited data**.
- LLMs use **transformer architecture**, are trained on **massive data** and can perform many tasks with **deep understanding**.
- LLMs generate more natural and human-like responses.
- **Examples:** Traditional NLP: TF-IDF, Word2Vec, BERT
LLM: GPT, LLaMA, Claude, Gemini.



4 Encoder vs Decoder

- **Encoder** reads and understands the input text and converts it into a numerical representation (embeddings).
- **Decoder** takes this representation and **generates** the output text, one token at a time.
- Some models use only **encoder** (e.g., BERT), some use only **decoder** (e.g., GPT) and some use both (e.g., T5, BART).



In Short:

- LLM** → A powerful AI model trained on **massive text data** to understand and generate human language.
- Why we need LLM** → Works across many tasks with deep understanding.
- LLM vs Traditional NLP** → LLMs are larger, more powerful and more flexible.
- Encoder vs Decoder** → Encoder understands, Decoder generates.



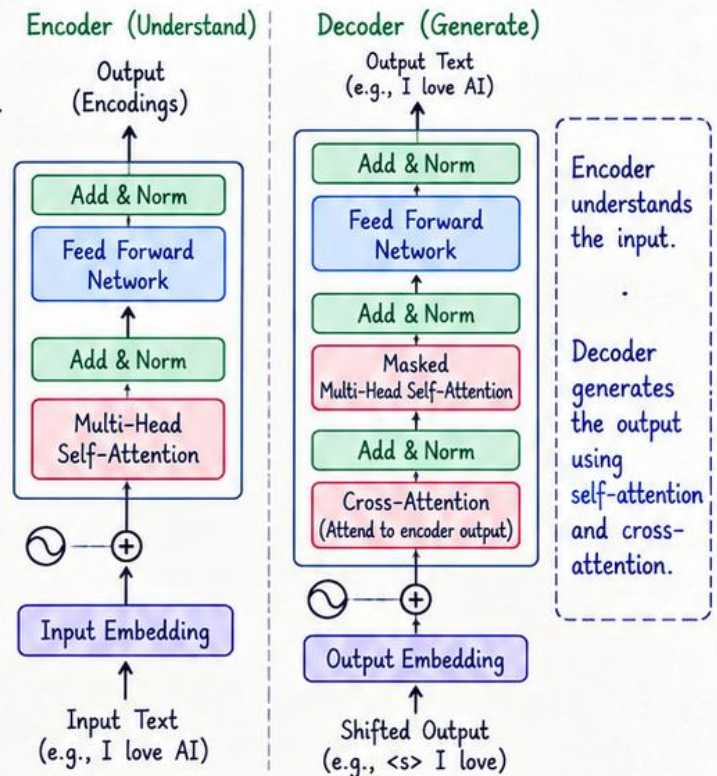
LLM Fundamentals (Part 2)



4 Transformer Architecture

- Transformer is a deep learning architecture introduced in 2017 (*Attention is All You Need*).
- It is based only on self-attention (no RNN or CNN).
- It has two main parts:
 - **Encoder** (understands the input)
 - **Decoder** (generates the output)
- Each layer consists of:
 - Multi-Head Self-Attention
 - Feed-Forward Network (FFN)
 - Layer Normalization
 - Residual Connections
- Uses positional encoding to understand the order of words.
- Can process all tokens in parallel, which makes it faster than RNNs.
- Examples:** BERT (encoder), GPT (decoder), T5 (encoder-decoder).

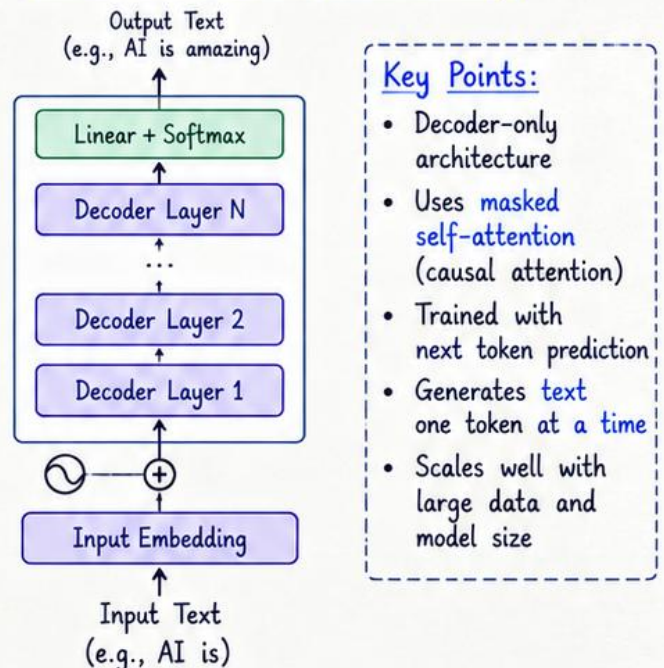
Transformer Architecture



5 GPT Architecture

- GPT (Generative Pre-trained Transformer) is a decoder-only transformer model.
- It uses causal (masked) self-attention, which means each token can only attend to previous tokens (not future tokens).
- It is trained on a large amount of text data using next token prediction.
- It generates text autoregressively (one token at a time).
- It consists of a stack of decoder layers, each with:
 - Masked Multi-Head Self-Attention
 - Feed-Forward Network (FFN)
 - Layer Normalization
 - Residual Connections
- Examples:** GPT-1, GPT-2, GPT-3, GPT-4, LLaMA (similar decoder-only architecture).

GPT Architecture (Decoder-only)



In Short:

Transformer Architecture → Encoder + Decoder, uses self-attention, can understand and generate text.

GPT Architecture → Decoder-only transformer, uses causal self-attention, generates text one token at a time.

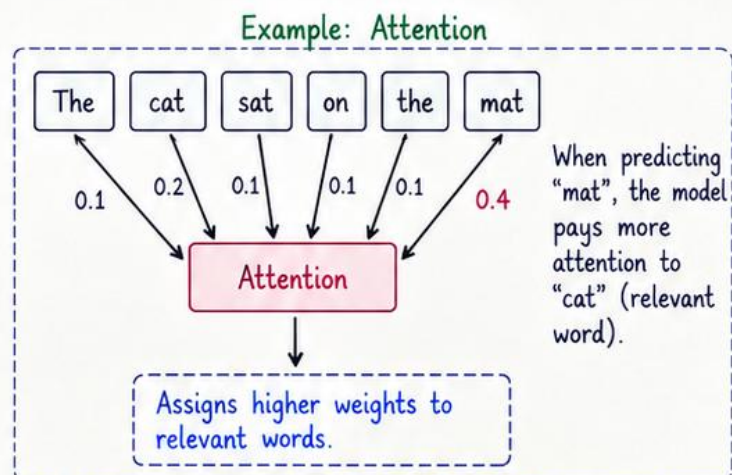


LLM Fundamentals (Part 3)



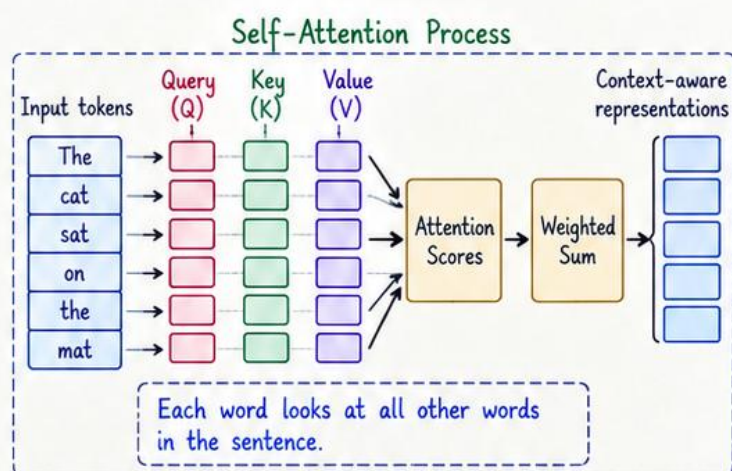
6 Attention Mechanism

- Attention helps the model focus on the **most relevant words** in a sentence when generating a word.
- Instead of treating all words equally, it assigns **higher importance** (attention scores) to useful words.
- It helps the model capture long-range dependencies (e.g., understand relationships between words far apart).
- **Examples:** machine translation, summarization, question answering etc.



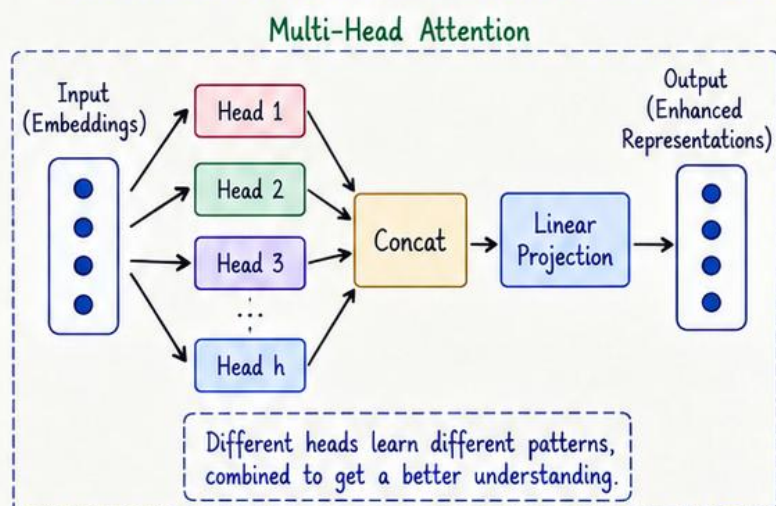
7 Self-Attention

- Self-Attention is a type of attention where each word in the input looks at **all the other words** in the same sentence.
- It calculates how much each word is related to other words and creates a **context-aware representation** for each word.
- It uses **Query (Q)**, **Key (K)** and **Value (V)** vectors to compute attention scores.
- **Examples:** Used in transformers, BERT, GPT etc.



8 Multi-Head Attention

- Multi-Head Attention runs **multiple self-attention heads** in parallel.
- Each head learns different relationships between words (e.g., syntax, coreference, meaning etc.).
- The outputs from all heads are combined to get a **richer representation**.
- It helps the model capture various types of information at the same time.
- **Examples:** Used in all modern LLMs like GPT, BERT, LLaMA etc.



In Short:

Attention Mechanism → Focus on important words using attention scores.

Self-Attention → Each word looks at all other words in the same sentence.

Multi-Head Attention → Multiple attention heads in parallel to capture different information.

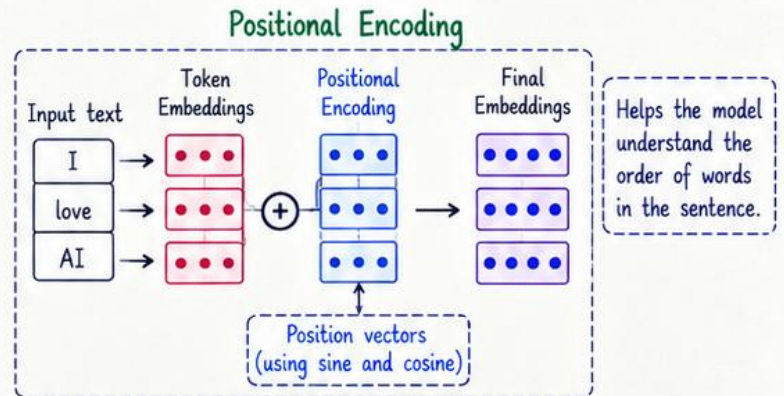


LLM Fundamentals (Part 4)



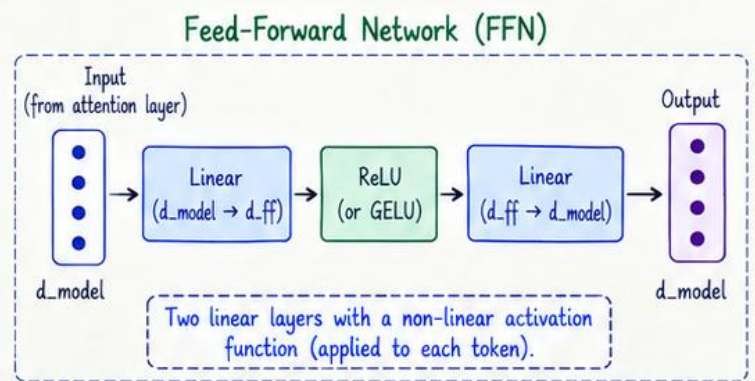
9 Positional Encoding

- Transformers do not have recurrence or convolution, so they **don't know the order of words**.
- Positional Encoding adds information about the **position** of each word to its embedding.
- It uses **sine and cosine functions** to create position vectors.
- These position vectors are added to the word embeddings.
- Examples:** Used in BERT, GPT, etc.



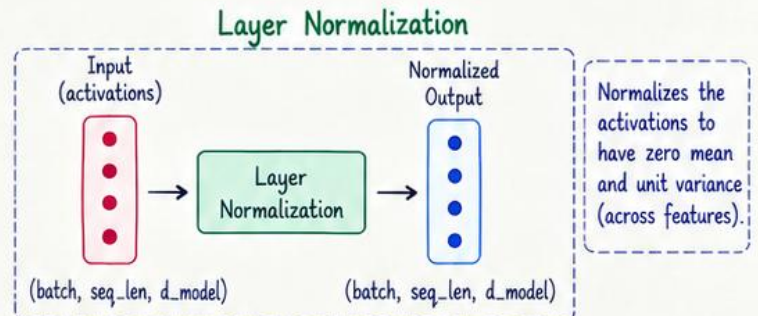
10 Feed-Forward Network (FFN)

- After the attention layer, each token goes through a Feed-Forward Network (FFN).
- It consists of **two linear transformation layers** with a **non-linear activation function** (ReLU or GELU).
- It is applied independently to each token (position-wise).
- It helps the model **learn complex patterns**.
- Examples:** Used in all transformer-based models.



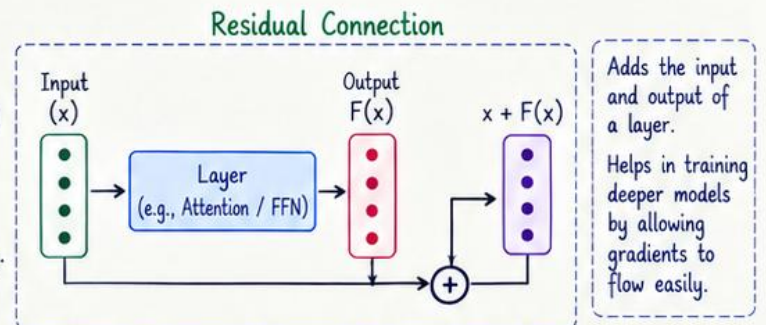
11 Layer Normalization

- Layer Normalization (LayerNorm) normalizes the activations across the features of each token.
- It helps to keep the **training stable** and faster.
- It reduces the problem of vanishing or exploding gradients.
- It is applied before or after attention and FFN layers.
- Examples:** Used in all modern LLMs.



12 Residual Connections

- Residual Connection (also called **skip connections**) add the **input** of a layer to its **output**.
- It helps the model **learn better and deeper** networks.
- It allows the gradient to flow easily during training.
- It reduces the problem of vanishing gradients.
- Examples:** Used around attention and FFN layers.



In Short:

Positional Encoding → Adds position information to embeddings (using sine & cosine).

Feed-Forward Network → Two linear layers with non-linear activation (ReLU/GELU).

Layer Normalization → Normalizes activations for stable and faster training.

Residual Connections → Adds input to output (skip connection) to help train deeper models.



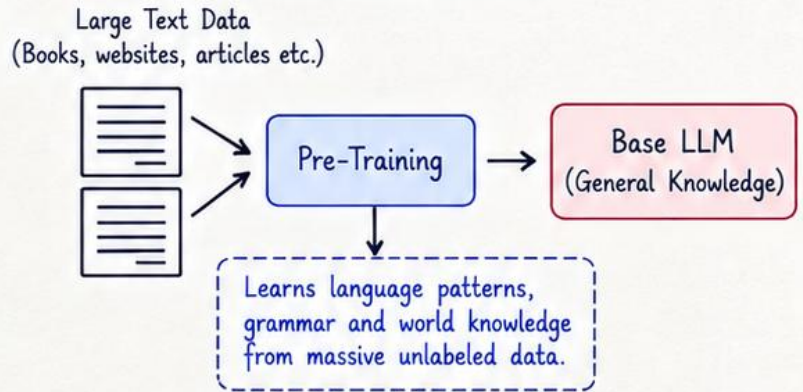
LLM Training (Part 1)



1 Pre-Training

- Pre-training is the first training stage where the model learns from massive amounts of unlabeled text data.
- The goal is to learn the patterns, grammar, facts and general world knowledge about language.
- It uses self-supervised learning (next token prediction).
- The model learns general language understanding, but it is not yet fine-tuned for specific tasks like chat, summarization etc.
- Examples: GPT, LLaMA, BERT etc. are first pre-trained on large datasets.

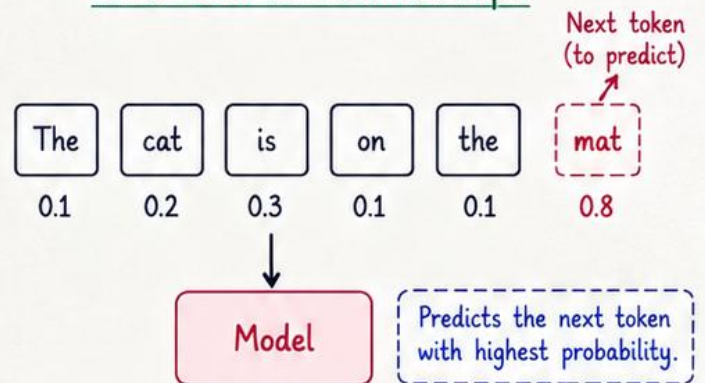
Pre-Training Process



2 Next Token Prediction

- Next token prediction means the model predicts the next word (token) in a sentence given the previous words.
- The model looks at the context (previous tokens) and calculates the probability of the next token.
- It is trained to minimize the difference between its predicted token and the actual next token.
- This simple task helps the model learn language patterns, grammar and meaning.
- Examples: Given "The cat is on the" → model predicts "mat".

Next Token Prediction Example



3 Training Data

- LLMs are trained on massive amounts of unlabeled text data from diverse sources.
- The data includes books, websites, articles, Wikipedia, code, forums, etc.
- Diverse and high-quality data helps the model learn better and reduces bias.
- The data is cleaned, filtered and tokenized before training.
- Examples: Common Crawl, Books, Wikipedia, GitHub code, research papers, news articles.

Sources of Training Data



In Short:

Pre-Training → First stage of training on massive unlabeled text data to learn general language knowledge.

Next Token Prediction → Model predicts the next token in a sentence.

Training Data → Large and diverse text data from books, websites, articles, code etc.



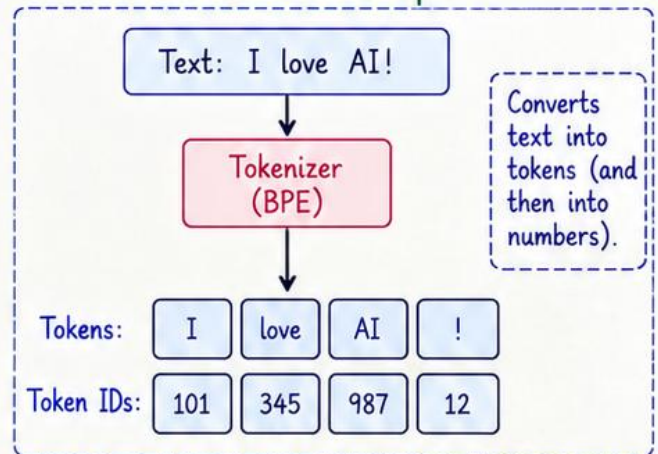
LLM Training (Part 2)



4 Tokenization

- Tokenization means breaking text into smaller pieces called **tokens**.
- A token can be a word, part of a word, a character or even a **symbol**.
- This helps the model convert text into **numbers (token IDs)** that it can understand.
- Modern LLMs use **subword tokenization** (e.g., Byte Pair Encoding - BPE) so that even new words can be handled.
- **Examples:** "unbelievable" → ["un", "believ", "able"]
"I love AI!" → ["I", "love", "AI", "!"]

Tokenization Example



5 Vocabulary

- Vocabulary is the list of all **unique tokens** that the model can understand.
- Each **token** in the vocabulary has a **unique ID (number)**.
- The vocabulary includes common words, subwords and **special tokens** (like <pad>, <unk>, <s>, </s> etc.).
- A larger vocabulary can cover more words but also increases the model size.
- **Examples:** ["the": 1, "cat": 2, "dog": 3, "<pad>": 0, "<unk>": 100]

Vocabulary Example

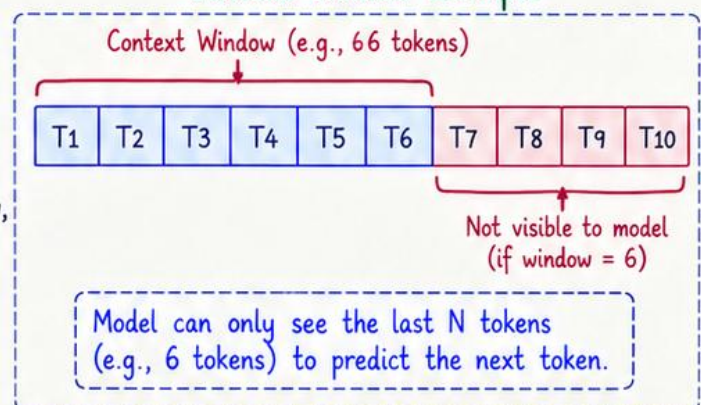
Token (Text)	Token ID
<pad>	0
the	1
cat	2
dog	3
AI	4
<unk>	100
...	...

Each token in the vocabulary has a unique ID.

6 Context Window

- Context window is the **maximum number of tokens** the model can look at in a single input.
- The model can use information only from these tokens to generate the next token.
- If the input is longer than the context window, older tokens are cut off (or handled using special techniques).
- A larger context window helps the model understand longer text, but requires more memory and computation.
- **Examples:** GPT-2 → 1,024 tokens, GPT-3 → 2,048 tokens, GPT-4 → 8,192 tokens (some models support even larger).

Context Window Example



In Short:

Tokenization → Breaks text into smaller tokens and converts them into numbers.

Vocabulary → List of all tokens with unique IDs.

Context Window → Maximum number of tokens the model can look at in a single input.



LLM Training (Part 3)



7 Loss Function

- A loss function measures how wrong the model's prediction is.
- It compares the predicted next token with the actual next token.
- The goal is to **minimize** this loss during training.
- A **smaller loss** means the model is making better predictions.
- Examples:** Cross-Entropy Loss (most common for LLMs).

Loss Function Example

Input: The cat is on the (predict next token)



mat	0.60
table	0.20
floor	0.10
bed	0.05
...	...

If the actual next token is "mat", the loss will be low. If the model assigns low probability to "mat", the loss will be high.

8 Cross-Entropy Loss

- Cross-Entropy Loss is a loss function used for classification problems.
- In LLMs, it measures the difference between the predicted probability distribution and the actual next token (true distribution).
- It gives a low loss when the correct token has high probability, and a high loss when the correct token has low probability.
- Formula** (for one token):

$$\mathcal{L} = - \sum_{i=1}^V y_i \log(\hat{y}_i)$$

where,

y_i = true distribution (1 for correct token, 0 for others)

$\hat{y}_i$ = predicted probability for token i

V = vocabulary size

Cross-Entropy Loss Example

Actual next token: "mat"

Token	Predicted Prob ($\hat{y}$)	True Prob (y)
mat	0.60	1
table	0.20	0
floor	0.10	0
bed	0.05	0
...	...	...

Since the correct token "mat", has high probability (0.60), the loss is low.

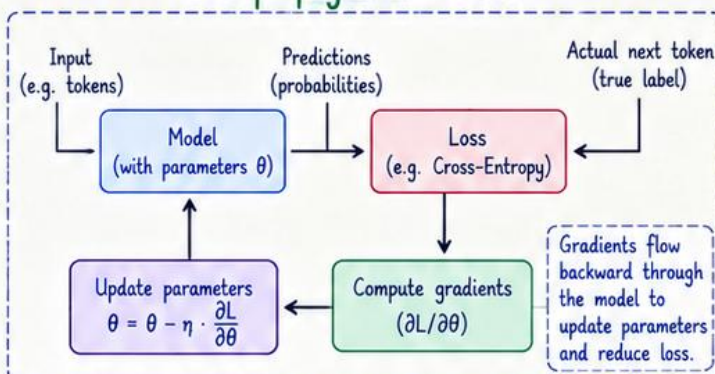
If the probability was low (e.g. 0.05), the loss would be high.

$$\text{Loss} = - \log(0.60) = 0.51$$

9 Backpropagation

- Backpropagation is the algorithm used to update the model's parameters during training.
- It calculates the **gradient** of the **loss** with respect to each parameter using the **chain rule**.
- The gradients tell us how to change the parameters to reduce the loss.
- The model's parameters are updated using an optimizer (e.g., Adam, SGD).
- This process is repeated many times until the loss becomes very small.

Backpropagation Process



In Short:

Loss Function → Measures how wrong the model's prediction is.

Cross-Entropy Loss → Common loss function used in LLMs to compare predicted and true distribution.

Backpropagation → Calculates gradients and updates model parameters to reduce the loss.



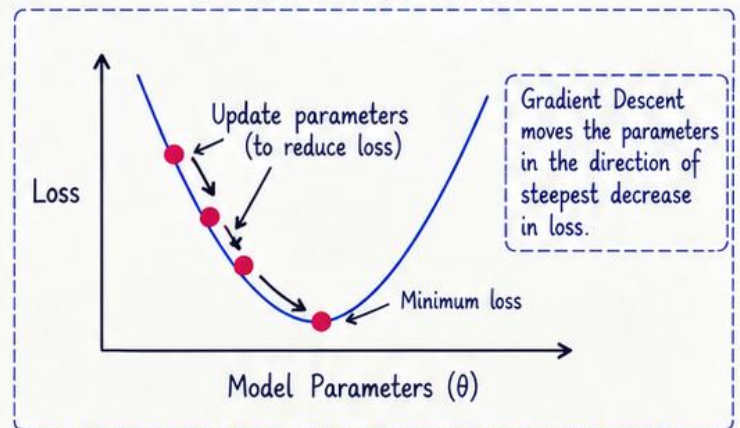
LLM Training (Part 4)



10 Gradient Descent

- Gradient Descent is an **optimization** algorithm used to **minimize the loss** function.
- It updates the model parameters in the direction of the **negative gradient** (to reduce the loss).
- The model takes small steps towards the values that give **lower loss**.
- This process is repeated many times until the loss becomes very small.
- Examples:** Used in training all neural networks, including LLMs.

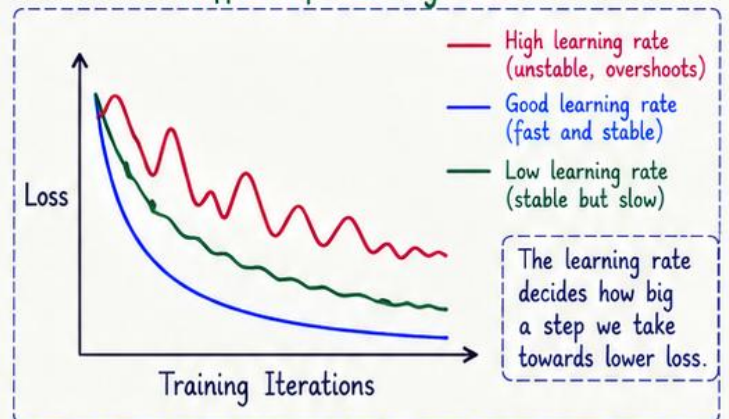
Gradient Descent Process



11 Learning Rate

- Learning Rate (α) controls how **big the steps** are while updating the parameters.
- A high learning rate can make training **unstable** (it may **overshoot** the minimum).
- A low learning rate makes training **stable but very slow**.
- The right learning rate helps the model **converge faster** and achieve a **lower loss**.
- Examples:** Common values: 0.1, 0.01, 0.001 (often tuned based on the task).

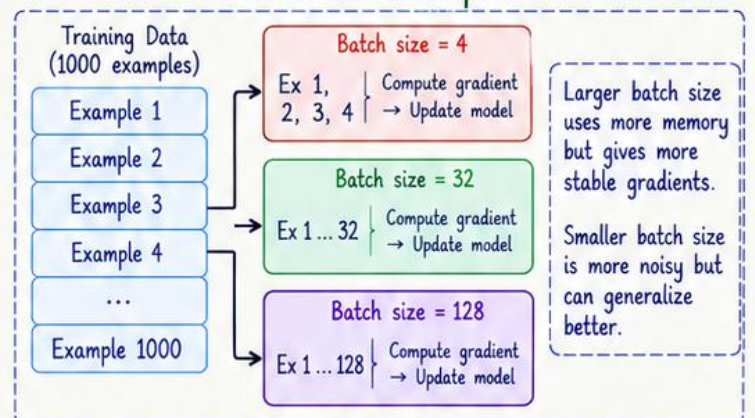
Effect of Learning Rate



12 Batch Size

- Batch Size is the number of training examples used to compute the gradient in one update step.
- A small batch size gives **noisier gradients** but can help the model **generalize better**.
- A large batch size gives **more stable gradients** but requires more memory and may generalize less well.
- The batch size affects training speed, memory usage and model performance.
- Examples:** Common values: 8, 16, 32, 64, 128.

Batch Size Example



In Short:

Gradient Descent → Updates model parameters to minimize the loss.

Learning Rate → Controls the size of the update steps.

Batch Size → Number of training examples used in one update step.



LLM Adaptation (Part 1)



1 Pre-Training vs Fine-Tuning

- Pre-training teaches the model general language knowledge using massive unlabeled data.
- Fine-tuning adapts the pre-trained model for specific tasks or domains using smaller, labeled data.
- Pre-training learns "general understanding", while fine-tuning learns "task-specific behavior".
- Fine-tuning is faster and cheaper than pre-training.

Pre-Training vs Fine-Tuning

Pre-Training

- Data: Massive unlabeled text (books, web, code)
- Goal: Learn general language understanding
- Cost: Very high
- Time: Long (weeks/months)
- Output: Base model (e.g., GPT, LLaMA)

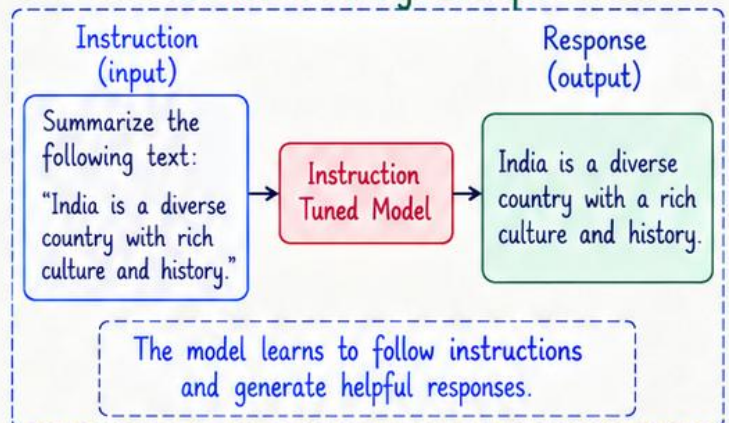
Fine-Tuning

- Data: Smaller labeled data (task/domain specific)
- Goal: Adapt to specific tasks (e.g., Q&A, summarization)
- Cost: Lower
- Time: Short (hours/days)
- Output: Task-specific model

2 Instruction Tuning

- Instruction Tuning teaches the model to follow natural language instructions.
- It uses instruction-response pairs (e.g., "Summarize this text" → "summary").
- Helps the model become more useful, helpful and aligned with human intent.
- Usually done using supervised fine-tuning on curated instruction datasets.
- Examples: Alpaca, FLAN, Dolly, OpenAI Instruct datasets.

Instruction Tuning Example



3 Supervised Fine-Tuning (SFT)

- Supervised Fine-Tuning (SFT) trains the model on labeled input-output pairs.
- The data contains a prompt (input) and a desired response (output).
- The model learns to generate responses similar to the given examples.
- SFT is the most common method for instruction tuning.
- Examples: Used to create models like ChatGPT, Claude, LLaMA-Instruct, etc.

SFT Training Example

Input (Prompt)	Output (Response)
What is AI?	AI is the simulation of human intelligence in machines.
Explain photosynthesis.	Photosynthesis is the process by which plants make food using sunlight, water and CO ₂ .
List 3 benefits of exercise.	1. Improves heart health 2. Boosts mood 3. Helps maintain weight

The model is trained on many such examples to learn the right response for a given prompt.



In Short:

Pre-Training → Learn general language knowledge from massive unlabeled data.

Instruction Tuning → Teach the model to follow natural language instructions.

Supervised Fine-Tuning (SFT) → Train on labeled input-output pairs to get task-specific behavior.

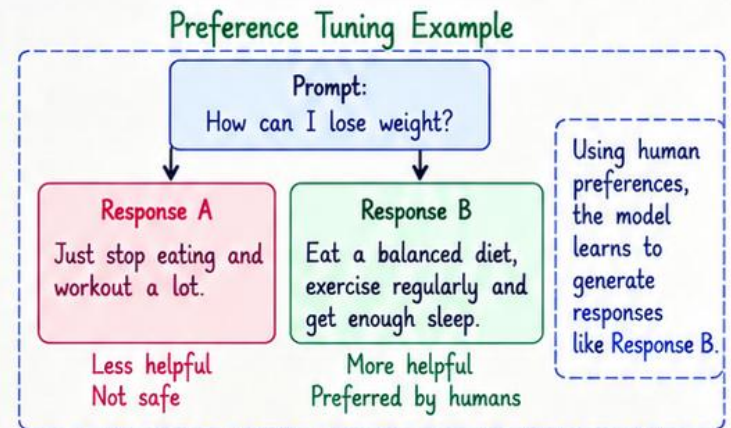


LLM Adaptation (Part 2)



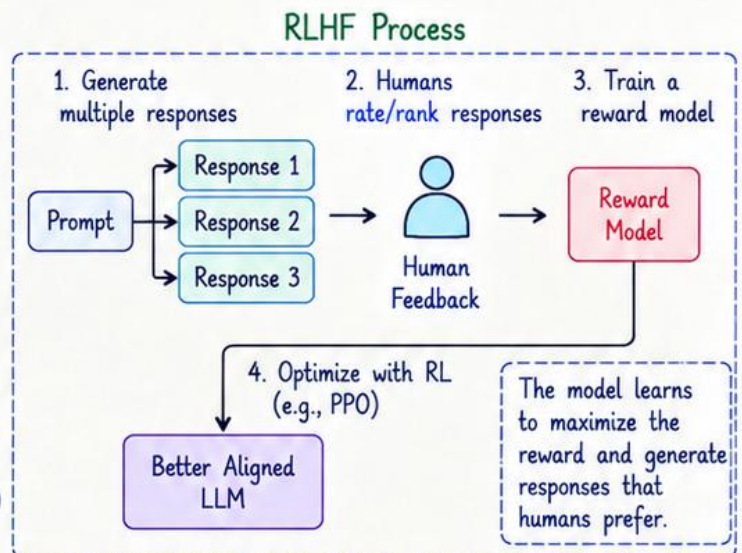
4 Preference Tuning

- Preference Tuning means training the model using **human preferences** instead of just **correct answers**.
- It teaches the model to generate responses that are more **helpful, safe and aligned** with **human values**.
- It can be done using methods like **RLHF** or **DPO**.
- **Examples:** Prefer a polite, helpful and factual answer over a rude or incorrect one.



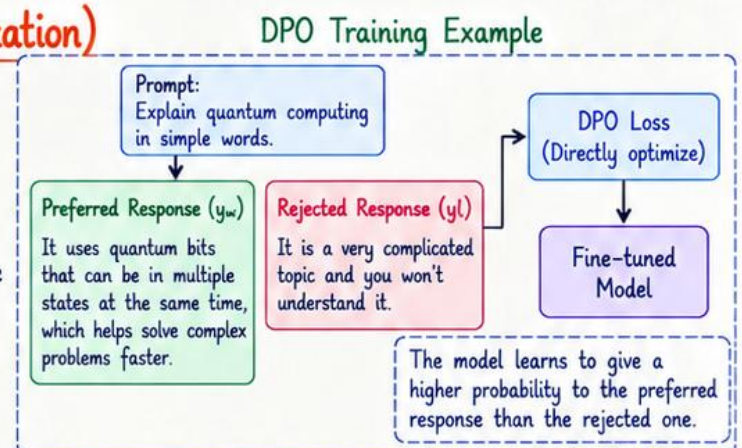
5 RLHF (Reinforcement Learning from Human Feedback)

- RLHF fine-tunes the model using **human feedback** and reinforcement learning.
- Humans rank or rate model responses.
- A **reward model** is trained on this feedback.
- The model is then optimized (using RL, e.g., PPO) to generate responses with higher rewards.
- Helps the model produce more **helpful, honest and safe** responses.
- **Examples:** ChatGPT, Claude, Bard (earlier versions) use RLHF.



6 DPO (Direct Preference Optimization)

- DPO is a simpler alternative to RLHF.
- It directly trains the model on pairs of preferred and rejected responses without using a reward model or complex RL.
- The model learns to give higher probability to the preferred response over the rejected one using a simple loss function.
- It is **easier, faster** and more stable than RLHF.
- **Examples:** Used in Llama 2, Mistral, Zephyr and many open-source models.



In Short:

Preference Tuning → Train using human preferences to generate helpful and safe responses.

RLHF → Use human feedback + reinforcement learning to optimize the model.

DPO → A simpler method that directly optimizes on preferred vs rejected responses (no reward model).



Fine-Tuning Fundamentals (Part 1)

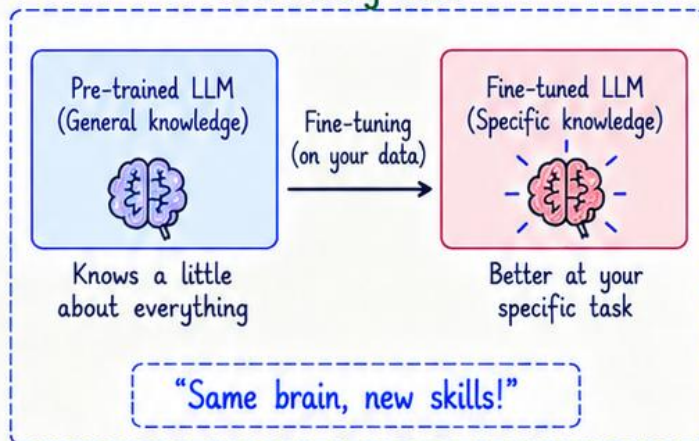
Adapting a pre-trained LLM for your specific needs



1 What is Fine-Tuning?

- Fine-tuning means further training a pre-trained model on a smaller, **task-specific dataset**.
- The base model already knows general language from massive data.
- We continue training it so that it learns the style, format and knowledge needed for a specific domain or task.
- It doesn't train from scratch → it just updates the **existing model weights** a little.
- **Result:** A model that performs better on your specific use case.

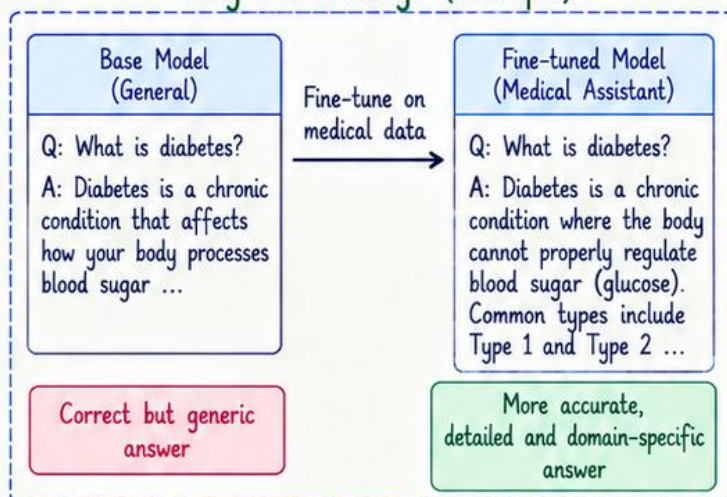
Fine-Tuning Idea



2 Why Fine-Tuning?

- To make the model perform better on a specific domain or task (e.g., medical, legal, customer support).
- To follow a **specific style, format or tone** (e.g., company writing style).
- To use your **own data** (private or internal knowledge).
- To get more accurate, relevant and consistent responses.
- Usually **cheaper** and **faster** than training a model from scratch.

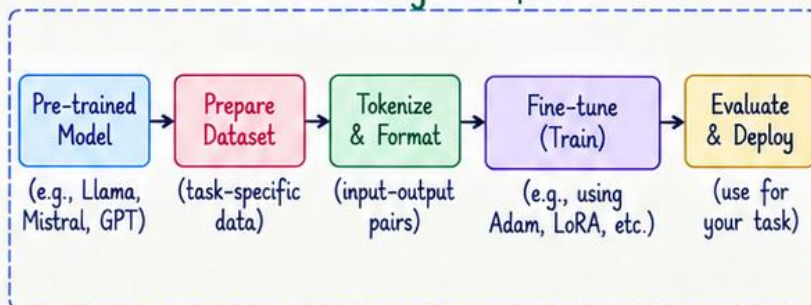
Why Fine-Tuning? (Example)



3 Fine-Tuning Workflow

- Start with a pre-trained model.
- Prepare a task-specific dataset (instruction-response pairs).
- Tokenize and format the data.
- Train (fine-tune) the model with an optimization algorithm (e.g., Adam).
- Evaluate the model on a validation set.
- Use the fine-tuned model for inference.

Fine-Tuning Workflow



In Short:

Fine-Tuning → Continue training a pre-trained model on your own data to make it better for a specific task.

Why → Improves accuracy, follows your domain/style, uses your data, and is cost-effective.

Workflow → Pre-trained model → prepare data → tokenize → train → evaluate → deploy.



Fine-Tuning Fundamentals (Part 2)

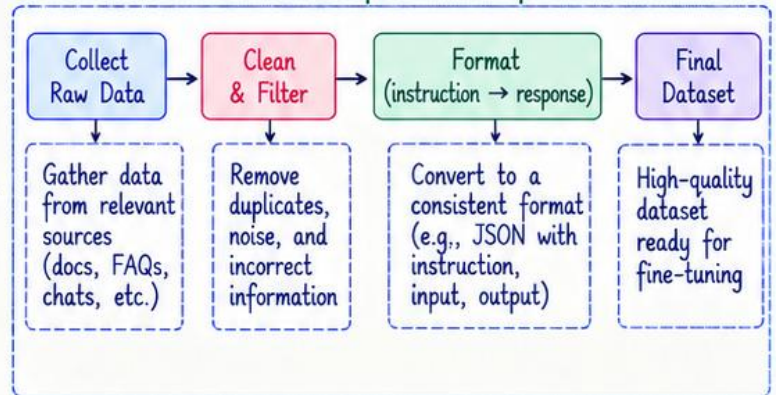
Preparing the right data for better fine-tuning

Data Drives Better Responses!

1 Dataset Preparation

- Dataset Preparation means collecting, cleaning and formatting data for fine-tuning.
- The data should be relevant to your task or domain.
- Remove noisy, duplicate or incorrect data.
- Ensure the data is **high-quality**, clear and consistent.
- The data is usually in **instruction → response** format.
- Examples:** Company FAQs, medical Q&A, customer support chats, coding examples, legal documents, etc.

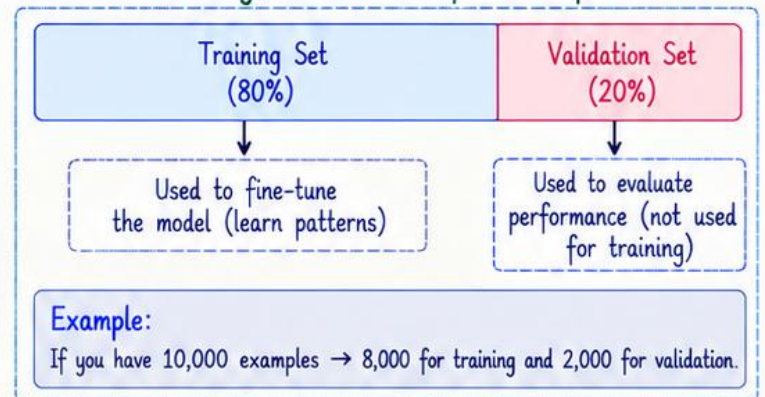
Dataset Preparation Steps



2 Training / Validation Split

- We split the dataset into **training** and **validation** sets.
- Training set is used to train (fine-tune) the model.
- Validation set is used to check how well the model is learning and to avoid overfitting.
- A common split is 80% training and 20% validation (can vary based on data size).
- Keep the split **random** and ensure both sets have similar type of data.

Training / Validation Split Example



3 Instruction Dataset

- An instruction dataset contains examples in **natural language instruction** format.
- Each example has an **instruction** (what to do) and a **response** (desired answer).
- It helps the model learn to follow instructions and generate helpful responses.
- Examples:** Alpaca, Dolly, OpenAI instruction dataset, or your own custom dataset.

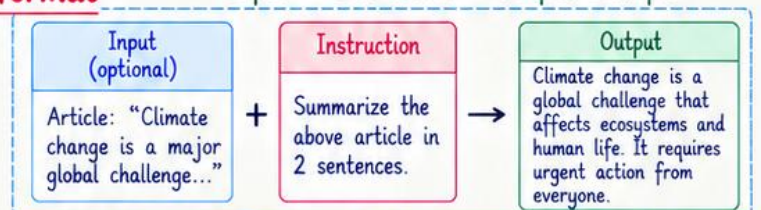
Instruction Dataset Example

Instruction	Input (optional)	Output (Response)
Summarize the text.	Text: "AI is transforming healthcare..."	AI is changing healthcare by ...
Translate to French.	Text: "Hello, how are you?"	"Bonjour, comment ça va ?"
Explain in simple words.	What is photosynthesis?	Photosynthesis is the process by which ...

4 Input → Instruction → Output format

- The most common format for fine-tuning is a structured format with three parts:
- Input (optional):** extra context or data
- Instruction:** what the model should do
- Output:** the expected response

Input → Instruction → Output Example



In Short:

Prepare **high-quality** data → Split into **training/validation** → Use **instruction** format → **Fine-tune** for better results.

Same
Big Model
Smarter
Fine-Tuning!

Parameter-Efficient Fine-Tuning (PEFT) (Part 1)

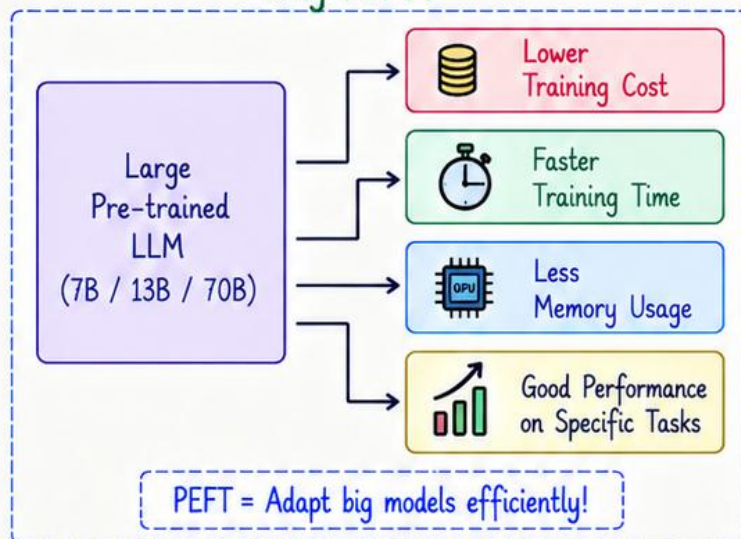
Big results with fewer parameters!



1 Why PEFT?

- Modern LLMs have billions of parameters, so full fine-tuning is expensive and resource-intensive.
- PEFT (Parameter-Efficient Fine-Tuning) allows us to adapt the model to specific tasks by training only a small number of additional parameters.
- It reduces compute cost, memory usage and training time while still giving high performance.
- Makes fine-tuning accessible on smaller hardware (even a single GPU).
- Examples:** LoRA, QLoRA, Adapter Tuning, Prefix Tuning, Prompt Tuning, etc.

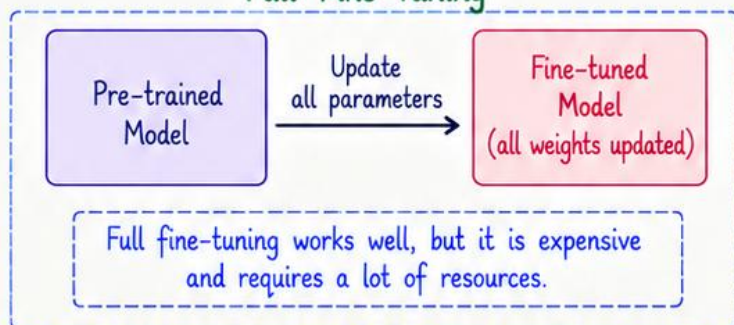
Why PEFT?



2 Full Fine-Tuning

- In full fine-tuning, we update all (or most of) the model parameters using task-specific data.
- The entire model is trained, which requires high GPU memory, longer training time and more compute resources.
- It can achieve very good performance, but it is costly and not always practical for large models.
- Examples:** Fine-tuning all weights of LLaMA, GPT, etc. on domain-specific data.

Full Fine-Tuning



3 Full Fine-Tuning Vs PEFT

- Full fine-tuning updates all model parameters, while PEFT updates only a small number of additional parameters (e.g., adapters, LoRA).
- Full fine-tuning gives high flexibility but is costly. PEFT is more efficient, faster and requires less memory.
- PEFT can achieve comparable performance to full fine-tuning for many real-world tasks.

Full Fine-Tuning vs PEFT

Aspect	Full Fine-Tuning	PEFT
Parameters Updated	All (or most)	Only a small number (e.g., LoRA, adapters)
Compute Cost	High	Low
Memory Usage	High	Low
Training Time	Long	Short
Performance	Very high (best)	Comparable (for many tasks)
Flexibility	High	Moderate (task-dependent)
Example Methods	Standard fine-tuning	LoRA, QLoRA, Adapters, Prefix Tuning, Prompt Tuning



In Short:

Full Fine-Tuning → Update all model parameters (high cost, high performance).

PEFT → Update only a small number of parameters (low cost, faster, and still good performance).

Same
Big Model
More
Possibilities!

Parameter-Efficient Fine-Tuning (PEFT) (Part 2)

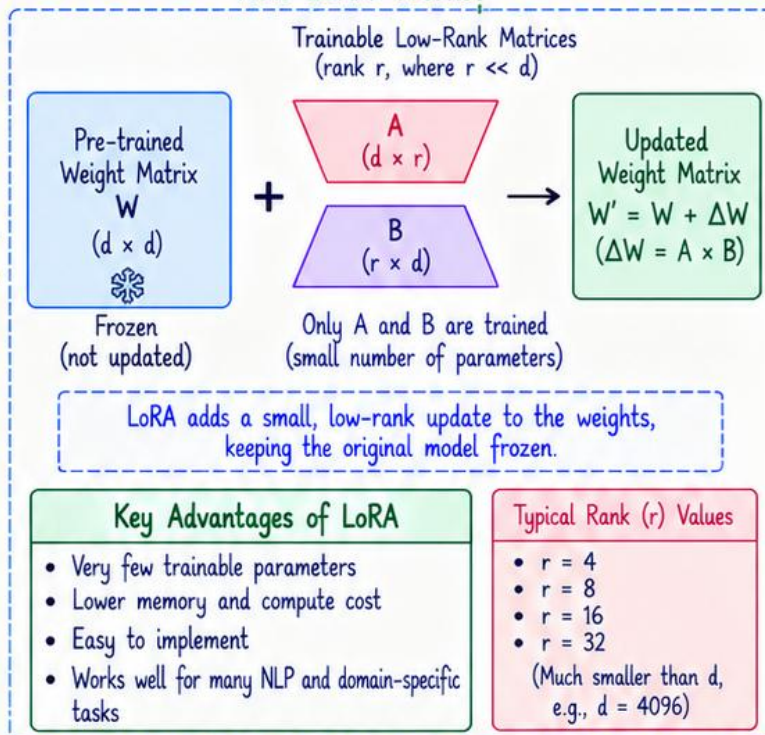
Efficient methods to adapt large models!

Less
Parameters
More Impact

4 LoRA (Low-Rank Adaptation)

- LoRA is a popular PEFT method.
- Instead of updating all model parameters, LoRA adds small, trainable low-rank matrices to the existing weights.
- The original model weights are kept frozen.
- Only the new low-rank matrices are trained, which greatly reduces the number of trainable parameters.
- After training, the low-rank update is combined with the original weights.
- It gives performance close to full fine-tuning but with much lower compute and memory cost.
- Examples:** Used for fine-tuning LLaMA, Mistral, GPT, etc.

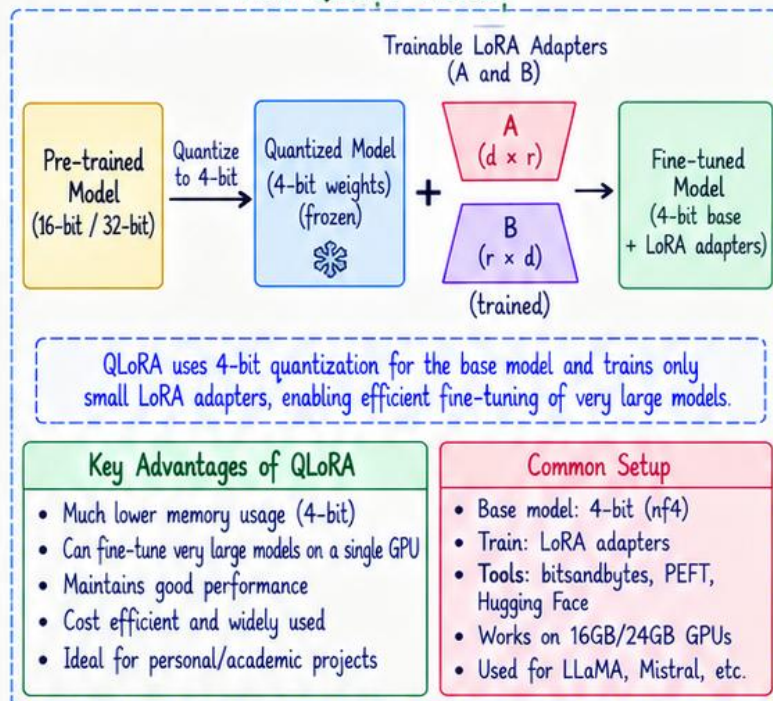
How LoRA Works?



5 QLoRA (Quantized LoRA)

- QLoRA is an improved version of LoRA that uses quantization.
- In QLoRA, the base model is loaded in 4-bit (or 8-bit) precision to reduce memory usage.
- The original model weights are quantized (and kept frozen).
- Like LoRA, only small low-rank matrices (A and B) are trained.
- This allows fine-tuning very large models (e.g., 7B, 13B, 70B) on a single GPU with limited memory.
- It maintains good performance while being much more memory and cost efficient.
- Examples:** Widely used for fine-tuning LLaMA, Mistral, Falcon, etc. on consumer GPUs (e.g., 16GB/24GB).

How QLoRA Works?



In Short:

LoRA → Adds small, trainable low-rank matrices to frozen weights.

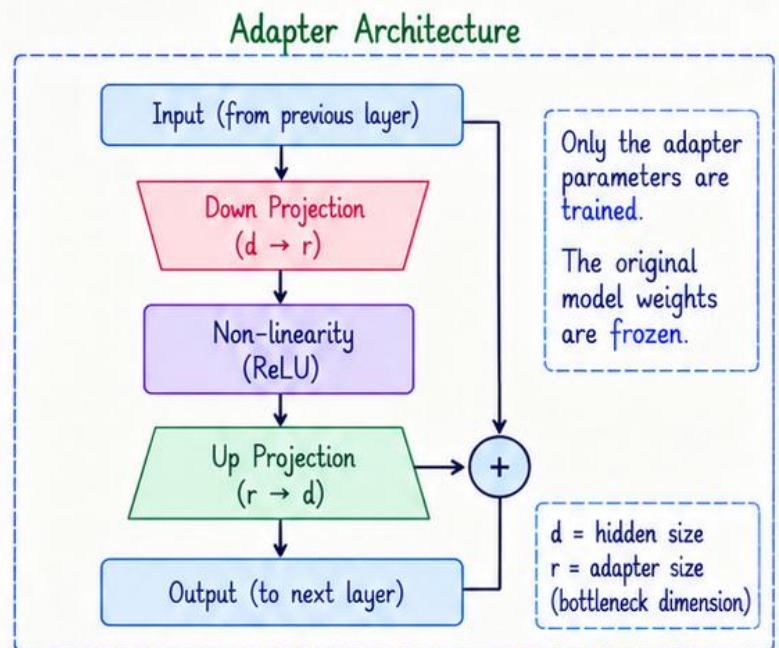
QLoRA → Uses quantization (4-bit) + LoRA to fine-tune very large models efficiently.

Same intelligence
Lower cost
Bigger possibilities!

Parameter-Efficient Fine-Tuning (PEFT) (Part 3)

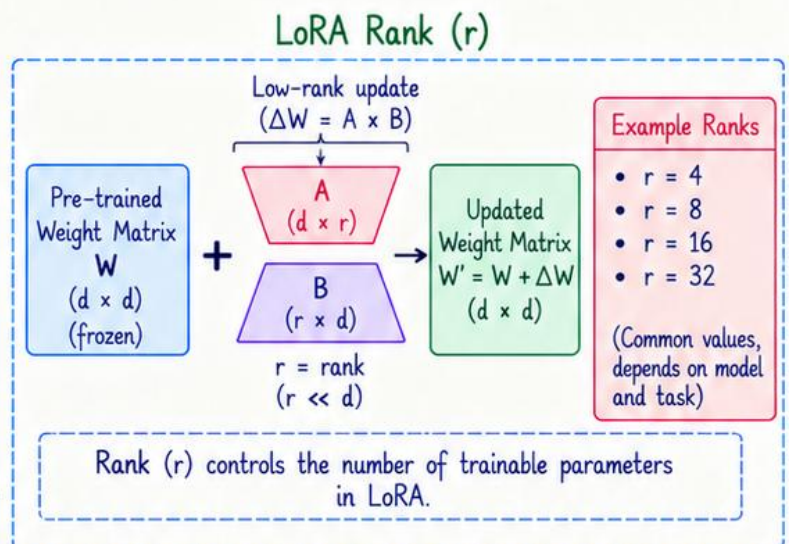
6 Adapters

- Adapters are small neural network modules added to each transformer layer.
- The original model weights are kept frozen, and only the adapter parameters are trained.
- The adapter module usually has a bottleneck structure (down-projection $\rightarrow$ non-linearity $\rightarrow$ up-projection).
- It adds a small number of parameters, so training is faster and more efficient.
- After training, we can keep multiple adapters for different tasks and switch between them without changing the base model.
- Examples:** Used in BERT, T5, LLaMA, etc. for domain-specific or task-specific fine-tuning.



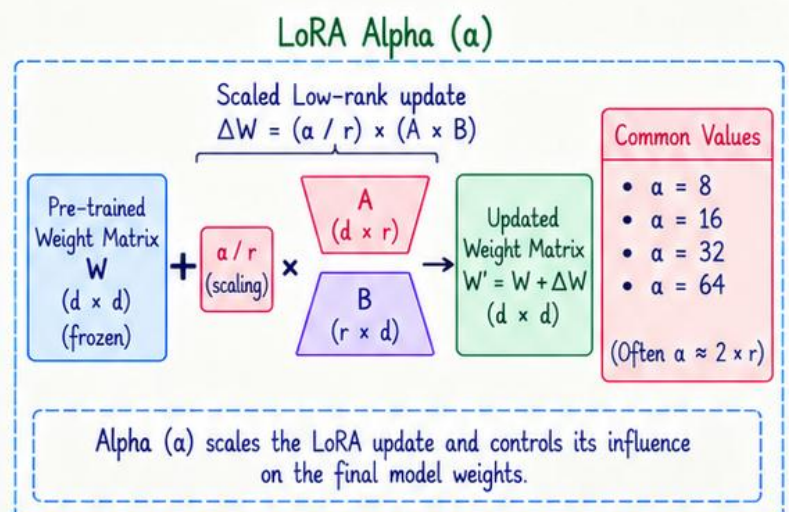
7 LoRA Rank (r)

- In LoRA, we use two low-rank matrices A and B to approximate the weight update ($\Delta W = A \times B$).
- The rank (r) is the size of the low-rank dimension.
- A smaller rank means fewer parameters, lower memory, and faster training.
- A larger rank gives more learnable capacity but uses more memory and compute.
- The rank is usually much smaller than the original hidden size (e.g., 4, 8, 16, 32).
- There is a trade-off between efficiency and performance.



8 LoRA Alpha

- LoRA Alpha (α) is a scaling factor applied to the LoRA update.
- The LoRA update is computed as:
$$\Delta W = (\alpha / r) \times (A \times B)$$
- Alpha controls how much influence the LoRA update has on the final weights.
- A larger alpha increases the impact of the LoRA update, while a smaller alpha reduces it.
- It helps to stabilize training, especially when using small ranks.
- Common values:** 8, 16, 32, 64 (usually $\alpha \approx 2 \times r$).

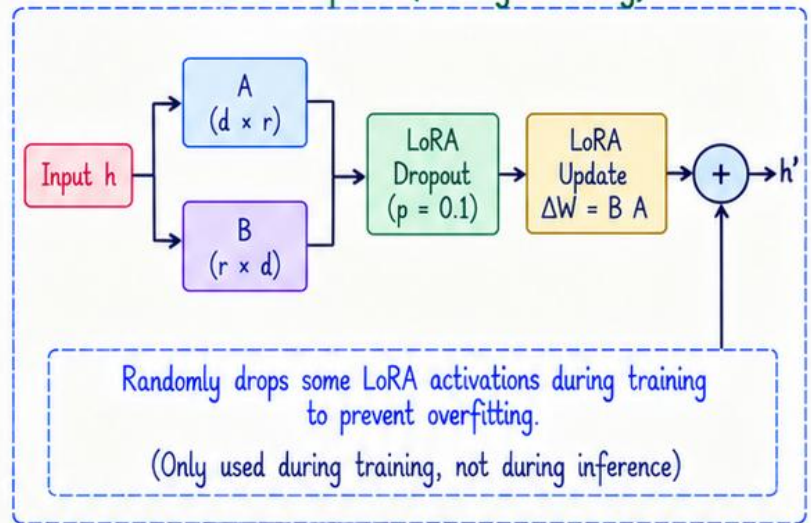


Parameter-Efficient Fine-Tuning (PEFT) (Part 3)

9 LoRA Dropout

- LoRA Dropout is a regularization technique applied to the LoRA update (A and B matrices).
- During training, a small fraction of the LoRA activations are randomly dropped (set to zero).
- It helps prevent overfitting and makes the model more robust.
- It is applied only to the LoRA layers, not to the original model weights.
- Common values:** 0.0, 0.05, 0.1 (0.1 is often used).

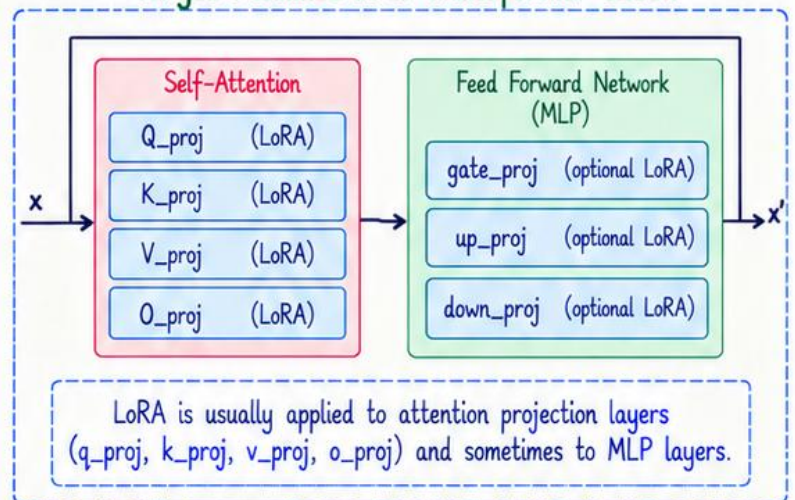
LoRA Dropout (During Training)



10 Target Modules

- In Transformer models, LoRA is not applied to all layers, but only to specific modules (target modules).
- Common target modules are the linear layers used for attention and sometimes in the feed-forward network.
- Typical target modules:** q_proj, k_proj, v_proj, o_proj (and sometimes gate_proj, up_proj, down_proj).
- Choosing the right target modules helps achieve good performance with fewer parameters and lower compute cost.

Target Modules in a Transformer Block



11 LoRA vs QLoRA

- LoRA uses full precision (e.g., 16-bit) base model weights.
- QLoRA uses a quantized base model (e.g., 4-bit) and LoRA adapters.
- QLoRA significantly reduces memory usage, allowing fine-tuning of very large models on a single GPU.
- Both methods train only a small number of additional parameters (LoRA adapters).

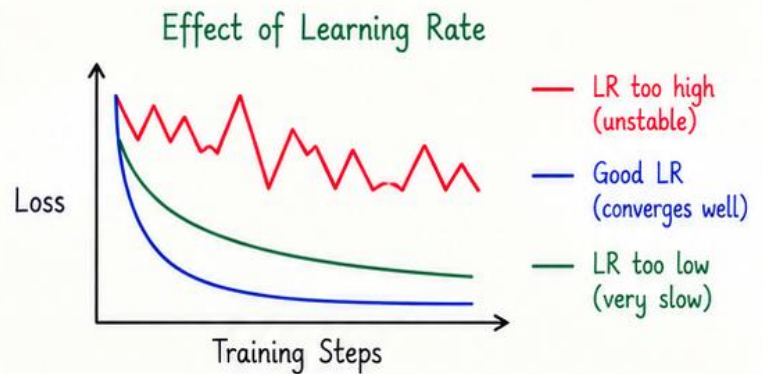
LoRA vs QLoRA (Comparison)

Aspect	LoRA	QLoRA
Base Model Precision	16-bit (FP16/BF16)	4-bit (quantized)
Trainable Parameters	LoRA adapters (only a few)	LoRA adapters (only a few)
Memory Usage	Higher	Much lower
Compute Cost	Higher	Lower
Model Size Supported	Medium to large	Very large (e.g., 70B)
Performance	Very good	Comparable to LoRA
Use Case	When you have enough GPU memory	When GPU memory is limited

Fine-Tuning Optimization (Part 1)

1 Learning Rate

- Learning rate (LR) controls how big of a step the model takes to update the weights during training.
- If LR is too high, the model may not converge (keeps overshooting).
- If LR is too low, training becomes very slow and may get stuck.
- A good learning rate helps the model learn faster and reach better performance.
- Common values: $1e-5$, $2e-5$, $5e-5$, $1e-4$ (depends on model and task).

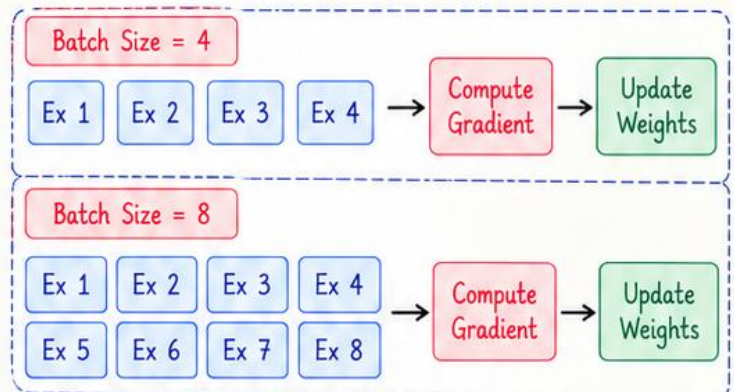


Learning rate decides how big a step to take towards the best weights.

2 Batch Size

- Batch size is the number of training examples used to compute the gradient before updating the model weights.
- A small batch size gives noisier gradients but can help generalization.
- A large batch size gives more stable gradients but requires more memory and may generalize less.
- The right batch size depends on your hardware (GPU memory) and task.
- Common values: 8, 16, 32, 64, 128.

Batch Size Example

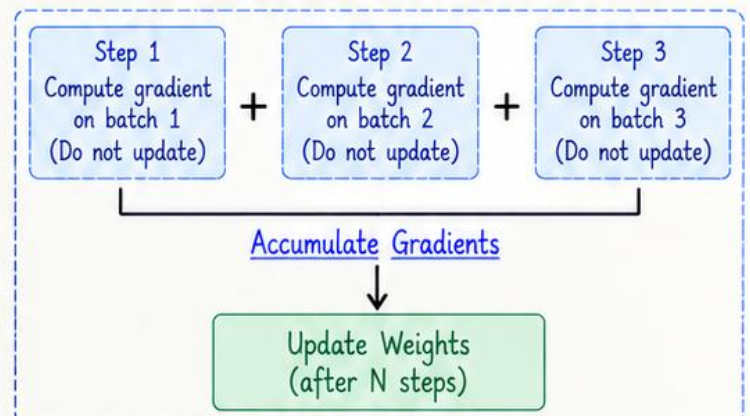


Batch size decides how many examples are used to compute the gradient at once.

3 Gradient Accumulation

- Gradient accumulation allows us to simulate a larger batch size using smaller batches.
- Instead of updating weights after every batch, we accumulate (sum) the gradients for multiple steps and update once.
- This is useful when GPU memory is limited.
- It gives similar results to a large batch size but with lower memory usage.
- Common values: 2, 4, 8, 16 (number of accumulation steps).

How Gradient Accumulation Works



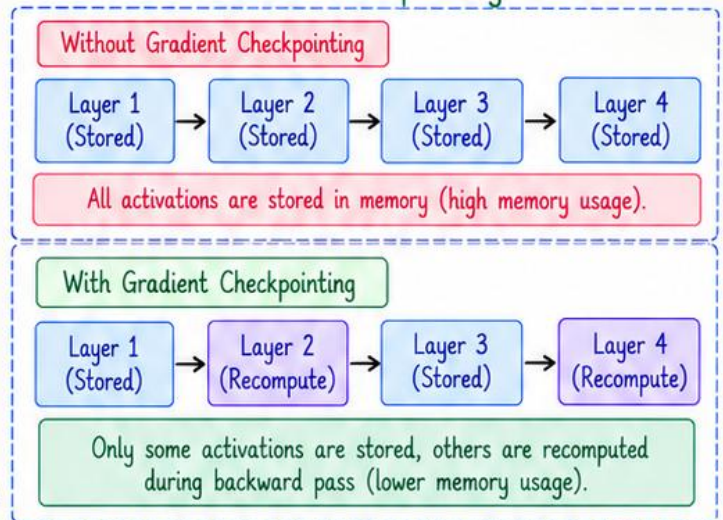
Helps train with a larger effective batch size when memory is limited.

Fine-Tuning Optimization (Part2)

4 Gradient Checkpointing

- Gradient checkpointing saves memory by not storing all intermediate activations during the forward pass.
- Instead, it only saves a few checkpoints and recomputes the missing activations during the backward pass.
- This reduces GPU memory usage at the cost of extra computation time.
- Very useful for training large models (e.g., LLMs) on limited GPU memory.
- Commonly used in libraries like Hugging Face (`use_gradient_checkpointing=True`).

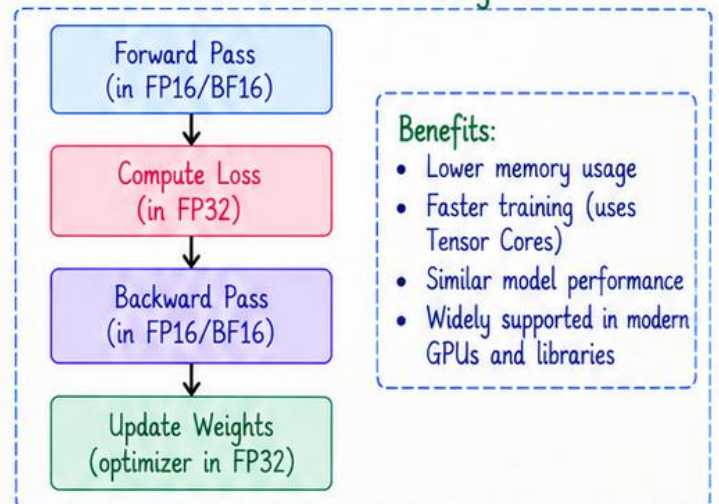
How Gradient Checkpointing Works



5 Mixed Precision

- Mixed precision training uses a combination of 16-bit (FP16 or BF16) and 32-bit (FP32) numbers.
- Model weights and activations are mostly in 16-bit, while some parts (like gradients or optimizer states) may remain in 32-bit for stability.
- It reduces memory usage, speeds up training and can work well with modern GPUs (e.g., NVIDIA A100, H100).
- Widely used in deep learning and LLM fine-tuning with frameworks like PyTorch (`autocast`, `GradScaler`) and Hugging Face (`TrainingArguments` with `fp16/bf16`).

Mixed Precision Training Flow



6 FP16 / BF16

- FP16 (Float16) uses 16 bits to represent numbers (1 sign bit, 5 exponent bits, 10 mantissa bits).
- BF16 (BFloat16) also uses 16 bits (1 sign bit, 8 exponent bits, 7 mantissa bits).
- FP16 has higher precision (more mantissa bits) but smaller range.
- BF16 has a larger range (same as FP32), making it more stable and easier to train.
- Both reduce memory usage and speed up training compared to FP32.
- Common practice:** use FP16 on older GPUs and BF16 on newer GPUs (e.g., A100, H100).

FP16 vs BF16

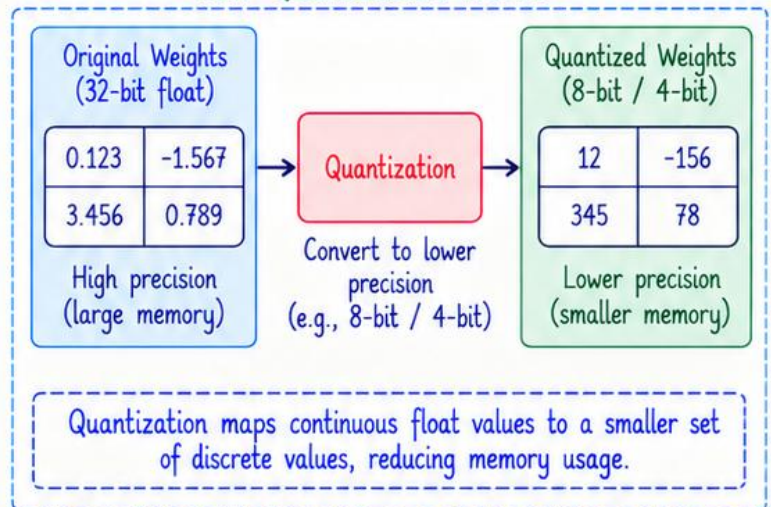
Aspect	FP16 (Float16)	BF16 (BFloat16)
Total Bits	16	16
Sign Bits	1	1
Exponent Bits	5	8
Mantissa Bits	10	7
Numeric Range	Smaller	Larger (same as FP32)
Precision	Higher (more precise)	Lower (less precise)
Training Stability	Can be unstable (sometimes needs loss scaling)	More stable
Common GPUs	Older GPUs (e.g., V100, T4, RTX series)	Newer GPUs (e.g., A100, H100)
Use Case	Good for inference and some training	Preferred for LLM training

Fine-Tuning Optimization (Part3)

7 Quantization

- Quantization reduces the precision of model weights and activations (e.g., from 32-bit to 8-bit or 4-bit).
- It significantly reduces memory usage and can speed up inference and training.
- It works because model weights can be represented with lower precision with minimal loss in performance.
- Widely used in LLM fine-tuning (especially with methods like QLoRA).
- Examples: 8-bit, 4-bit, GPTQ, AWQ.

How Quantization Works



8 4-bit / 8-bit

- 8-bit quantization uses 8 bits to represent each weight (256 possible values).
- 4-bit quantization uses 4 bits (16 possible values), giving even smaller model size.
- 4-bit saves more memory but may have a slightly larger impact on performance compared to 8-bit.
- 8-bit is a good balance between efficiency and performance.
- 4-bit is commonly used with QLoRA for fine-tuning very large models on a single GPU.

4-bit vs 8-bit Comparison

Aspect	8-bit	4-bit
Number of values	256 (2^8)	16 (2^4)
Memory usage	~ 2x smaller than 16-bit	~ 4x smaller than 16-bit
Model size	Smaller	Much smaller
Performance	Very close to full precision	Slightly lower (but often acceptable)
Use case	General purpose fine-tuning / inference	Large models (e.g., 7B, 13B, 70B) with limited GPU memory
Examples	BitsAndBytes 8-bit, LLM.int8()	QLoRA (4-bit), NF4, GPTQ (4-bit)

9 GPU Memory Optimization

- Fine-tuning large models requires careful memory management.
- We can combine multiple techniques to fit the model in limited GPU memory.
- Key techniques: quantization, gradient checkpointing, mixed precision, smaller batch size, gradient accumulation, PEFT (LoRA/QLoRA).
- Also free unused memory during training and use efficient data loading.
- These optimizations allow fine-tuning very large models on consumer GPUs (e.g., 16GB/24GB).

Common Techniques to Save GPU Memory

Technique	What it Does
Quantization (4-bit / 8-bit)	Reduces model size and memory usage.
Gradient Checkpointing	Saves memory by recomputing activations during backward pass.
Mixed Precision (FP16 / BF16)	Uses lower precision to reduce memory usage.
Smaller Batch Size	Uses fewer examples per forward pass.
Gradient Accumulation	Simulates larger batch size without increasing memory.
PEFT (LoRA / QLoRA)	Trains only small adapter parameters instead of full model.
Efficient Data Loading	Uses optimized dataloaders and smaller sequence length.
Free Unused Memory	Clear cache and remove unused variables (e.g., torch.cuda.empty_cache()).